

OBJECTIFS PEDAGOGIQUES :

- Utiliser les structures de données (tableaux et enregistrements) ;
- Ecrire l'algorithme de recherche séquentielle ;
- Résoudre des problèmes concrets de recherche des données dans un tableau ;

CONTROLE DE PRESREQUIS :

1. Définir algorithme, algorithmique, instruction, variable, compteur
2. Donner la structure minimale d'un algorithme
3. Donner la syntaxe de déclaration d'une variable et celle d'une constante

SITUATION PROBLEME :

Un lycée de la place voudrait une application qui permet de gérer les notes des élèves. Celui qui a la charge du travail ne sait comment faire pour gérer les notes des élèves surtout quand le nombre d'élèves est très grand. Il ne peut pas exemple déclarer 40 variable pour un effectif de 40 élèves. Il vous demande de l'aide pour la conception de l'algorithme.

CONSIGNES :

1. Qu'est-ce qu'une variable ? (**Réponse(s) attendue(s)** : suite finie ordonnée d'instruction permettant de résoudre un pb)
2. Peut-on utiliser type de variable de base pour gérer les notes de 40 élèves par exemple ? Sinon existe-t-il une autre solution ? si oui donner cette solution (**Réponse(s) attendue(s)** : Non. Oui il existe une autre solution qui est l'utilisation des structures de données ou créer des types de données adaptés)

RESUME

Une **structure de données** est un type de variable fabriquée à partir d'autres types de variables (généralement les types de base). On peut citer comme exemple de structure données : **Les tableaux, Les enregistrements, Liste, Pile, File d'attente** ou **Queue** (anglais). Mais dans cette leçon nous nous concentrerons sur les **Les tableaux, Les enregistrements**

I- LES ENREGISTREMENTS

Il est souvent pratique de regrouper logiquement plusieurs variables en une seule variable composée. On parle alors de structure ou d'enregistrement. Un type enregistrement ou type structuré est un type **T** de variable **V** obtenu en juxtaposant plusieurs variables **V₁; V₂; ...** ayant chacune un type **T₁;T₂; ...**

- Les différentes variables **V_i** sont appelées champs de **V**
- Elles sont repérées par un identificateur de champ
- Si **v** est une variable de type structuré **T** possédant le champ **ch**, alors la variable **v.ch** est une variable comme les autres : (**type, adresse, valeur**).

La déclaration de type enregistrement se fait de la manière suivante :

```
nom_de_type = Enregistrement
                nom_du_champ_1 : type_du_champ_1;
                nom_du_champ_2 : type_du_champ_2;
                ...
            Fin ;
```

La déclaration de variable structurée : **V: nom_de_type;**

NB : La déclaration d'un type enregistrement dans la partie de la déclaration des variables juste après celle des constantes.

Exemple : Déclaration d'un type enregistrement permettant de stocker les informations sur un élève caractérisées par le nom, prenom, note et sexe.

```
Eleve = Enregistrement
                nom : chaine;
                prenom : chaine;
                sexe : caractere ;
                note : Reel ;
            Fin ;
```

Déclaration d'une variable de type **Eleve** : **Var e : Eleve ;**

Utilisation d'une variable de type **Eleve** : **e.note← 15 ;**

Lire (e.nom) ; Ecrire("nom élève :", e.nom) ;

II- LES TABLEAUX

Un tableau est une structure de donnée **T** qui permet de stocker un certain nombre d'éléments **T[i]** repérés par un index **i**. Les tableaux vérifient généralement les propriétés suivantes :

- Tous les éléments ont le même type de base ;
- Le nombre d'éléments (taille du tableau ou dimension) stockés est fixé.

La déclaration d'un tableau se fait de la manière suivante :

Nom_tableau : **Tableau** [indice_min...indice_max] de type_éléments_tableau ;

On peut effectuer plusieurs opérations sur un tableau.

1- Ajout et modification des éléments d'un tableau

Le tableau ci-dessous résume les opérations d'ajout et de suppression des éléments dans un tableau

Ajout et modification d'un élément dans tableau
Nom_tableau[indice] $\leftarrow$ valeur ou bien Lire (Nom_tableau[indice]); Ecrire ("nom élève :", e.nom) ; Exemple : Ajout des éléments dans un tableau d'entiers note . note [1] $\leftarrow$ 12; Ecrire (note [1]) ; Lire (note [2])) ; Ecrire (note [2]) ;

2- Recherche d'un élément dans un tableau

Le problème est de rechercher des informations dans un tableau. Dans la suite, on supposera que les éléments du tableau sont les nombres. On distingue généralement deux types de recherche dans tableau, à savoir la recherche séquentielle et la recherche dichotomique.

 Recherche dichotomique

Cette méthode s'applique si le tableau est déjà trié et s'apparente alors à la technique « **Diviser pour Régner** ».

 Recherche séquentielle

Cette méthode simple consiste à parcourir le tableau à partir du premier élément, et à s'arrêter dès que l'on trouve l'élément cherché (on ne cherche pas toutes les occurrences d'un élément).

Soient **T** un tableau de **n** éléments et **m** l'élément qu'on recherche.

Algorithme recherche séquentielle
Données : Un tableau T de n éléments et un élément m Résultat : Le premier indice i où se trouve l'élément m s'il est dans T , et sinon la réponse « Élément introuvable ! » i =1; Tantque (i <=n ET T [i]<> m) faire i $\leftarrow$ i +1; FinTanque


```
Si (i<=n) alors  
    Ecrire(" Elément en position", i) ;  
    Sinon  
        Ecrire(" Elément introuvable !") ;  
FinSi
```

- Les boucles **répéter... jusqu'à** et **tant que... faire** sont ce qu'on appelle des **boucles conditionnelles**.

Exemple : Appliquons l'algorithme de la recherche séquentielle sur tableau ci-dessous :

T	12	4	6	10	7	9	17
----------	-----------	----------	----------	-----------	----------	----------	-----------

m=7, n=7

Exécution de l'algorithme de la recherche séquentielle :

I	1	2	3	4	5	6	7
T	12	4	6	10	7	9	17
i<=n ET T [i]<>m	vrai	vrai	vrai	vrai	faux		

Résultat : " Elément en position", 5

SITUATION D'INTEGRATION :

Une école maternelle voudrait une petite application qui permettra de gérer les notes des élèves. Les notes de ces élèves sont des nombres entiers positifs. L'application doit être capable d'ajouter, afficher et rechercher les notes des élèves. On vous demande de l'aide sur l'algorithme.

- 1- Définir le terme **structure de données**
- 2- Pourquoi crée-t-on d'autres types de variable (structures de données) en algorithmique ?
- 3- Donner un différence entre **tableau** et **enregistrement**
- 4- Quelle est la structure de données la plus adaptée pour le stockage des notes des élèves en une seule fois ? Justifiez votre réponse
- 5- Dans la suite on suppose que les notes des élèves sont stockées dans un tableau **T**.

- 5.1** Donner l'instruction qui permet de déclarer le tableau **T** de taille **N**
- 5.2** Donner le code qui permet de lire (ajouter) tous éléments de **T** sachant que **N=50**
- 5.3** Donner le code qui permet d'afficher les notes de tous les élèves d'une salle. Ces notes se trouve dans le tableau **T**.

- 6- Donner la portion de code qui permet de créer une nouvelle structure qui sera chargée d'enregistrer les enfants de cette école sachant qu'un enfant est caractérisé par son nom, son sexe et son âge. Cette structure s'appellera « **Enfant** »

REINVESTISSEMENT

Une commune de la place voudrait un petit système qui sera chargé de gérer l'enregistrement des mototaximens. Ce système doit être capable de retrouver un mototaximen à partir de son matricule. Un mototaximen est caractérisé par son **nom**, **sexe** et **matricule**. Le nombre étant très grand, celui qui a la charge de mettre en place ce application à décider de créer un tableau contenant les mototaximens (ils sont 300). Il est bloqué et ne sait comment gérer un tel tableau et il vous demande de l'aide.

- 1- Quelle est la condition pour qu'on puisse appliquer un recherche dichotomique sur un tableau ?
- 2- Donner l'instruction permettant de créer un type qui sera chargé de stocker un mototaximen. Vous l'appellerez **Mototaximen**
- 3- Donner l'instruction qui permet de déclarer le tableau qui contiendra les mototaximens. Vous l'appellerez **Tab** de taille **N**.
- 4- Ecrire le code qui permet d'enregistrer (ajouter) les mototaximen une seule fois dans le tableau.
- 5- Modifier l'algorithme de la recherche séquentielle de tel sorte qu'on puisse rechercher un mototaximen à partir de son **matricule** et affiche son nom et son matricule. Exemple : si on recherche **Baba Simon**, de matricule **234543** et qu'on le trouve on affichera :
Nom : **Baba Simon**
Matricule : **234543**

0 **UNITE D'ENSEIGNEMENT 30 : ALGORITHME DE TRI ET RECHERCHE DANS UN TABLEAU**

OBJECTIFS PEDAGOGIQUES :

- Ecrire un algorithme de recherche séquentielle
- Exécuter pas à pas l'algorithme de tri par insertion
- Résoudre un problème concret de tri et de recherche des données dans un tableau

PREREQUIS

1. Définir tableau
2. Donner la syntaxe de déclaration d'un tableau
3. Ecrire un algorithme qui affiche tous les éléments d'un tableau

SITUATION PROBLEME

On Veut rechercher les noms d'un candidat à l'examen dans la liste des admis. Sachant que cette liste a été stockée dans un tableau, on désire écrire un algorithme qui effectuera automatiquement cette tâche et classera par ordre alphabétique les noms des candidats contenus dans ce tableau

CONSIGNES

- 1- Présentez quelques types de recherche dans un tableau que vous connaissez

(**Réponse attendue** : la recherche séquentielle, la recherche dichotomique)

- 2- Ecrire un algorithme permettant de rechercher un élément en parcourant un tableau (**Réponse attendue** : *Algo recherche séquentielle*)

Recherche séquentielle dans un tableau trié.

Fonction rechercheSeqTrie (tab : Tableau[0..MAX+1]
d'Éléments, x : Élément): Naturel

Déclaration i : Naturel

Début

i ← 0

Tant que x > tab[i] **faire**

i ← i+1

fintantque

Retourner i

Fin)

- 3- A quoi sert le tri des éléments dans un tableau ? (**Réponse attendue** : d'organiser une collection d'objets selon une relation d'ordres déterminée)

4- Présentez quelques types de tri dans un tableau (**Réponse attendue** : tri par insertion, tri rapide, tri à bulle)

5- Ecrire un algorithme de tri par insertion (**Réponse attendue**:

- a) **procédure tri_insertion(T)**
- b) **N** taille de T
- c) **Pour** i de 1 à N $\leftarrow$ 1 **faire**
- d) **x** $\leftarrow$ T[i]
- e) **j** $\leftarrow$ i
- f) **Tant que** j > 0 et T[j $\leftarrow$ 1] > x **faire**
- g) T[j $\leftarrow$ 1] T[j]
- h) **j** j $\leftarrow$ 1
- i) **_n** tant que
- j) T[j] $\leftarrow$ x
- k) **Finpour**
- l) **_fin** procédure)

RESUME

QUELQUES EXEMPLES D'ALGORITHMES DE RECHERCHE

La recherche d'un élément dans un tableau consiste à trouver sa position et le classer par ordre croissant, décroissant ou alphabétique, cependant, on distingue les types de recherches suivants

1. La recherche séquentielle ou recherche linéaire

C'est un algorithme pour trouver une valeur dans une liste. Elle consiste simplement à considérer les éléments de la liste les uns après les autres, jusqu'à ce que l'élément soit trouvé, ou que toutes les cases aient été lues. Elle est aussi appelée **recherche par balayage**.

► **Principe** : La recherche séquentielle consiste à prendre les éléments de la liste les uns après les autres, jusqu'à avoir trouvé la cible, ou avoir épuisé la liste

Exemple d'algorithme

Fonction rechercheSeqTrie (tab : Tableau [0..MAX+1] d'Éléments, x : Éléments)
: Naturel

Déclaration i : Naturel

Début

i $\leftarrow$ 0

Tant que x > tab[i] **faire**

i $\leftarrow$ i+1

fintantque

Retourner i
Fin.

2. La recherche dichotomique

La **recherche dichotomique**, ou **recherche par dichotomie** (en anglais : *binary search*) : c'est un algorithme de recherche pour trouver la position d'un élément dans un tableau trié.

► **Principe** : Le principe est le suivant : comparer l'élément avec la valeur de la case au milieu du tableau si les valeurs sont égales, la tâche est accomplie, sinon on recommence dans la moitié du tableau pertinente. L'algorithme est le suivant : Trouver la position la plus centrale du tableau (si le tableau est vide, sortir). Comparer la valeur de cette case à l'élément recherché ;

► **Description de l'algorithme** ... Si la valeur est égale à l'élément, alors retourner la position, sinon reprendre la procédure dans la moitié de tableau pertinente. On peut toujours se ramener à une moitié de tableau sur un tableau trié en ordre croissant. Si la valeur de la case est plus petite que l'élément, on continuera sur la moitié droite, c'est-à-dire sur la partie du tableau qui contient des nombres plus grands que la valeur de la case. Sinon, on continuera sur la moitié gauche.

Exemple d'algorithme

Fonction rechercheDicoTrie (tab : Tableau[0..MAX] d'Éléments, x : Élément)
:Naturel

Déclaration gauche,droit,milieu : Naturel

Début

Gauche←0;droit←MAX

Tant que gauche ≤ droit faire

Milieu← (gauche+droit) div 2

Si x=tab [milieu] alors retourner milieu finsi

Si x<tab [milieu] alors

Droit ←milieu-1

Sinon

Gauche ←milieu+1

Finsi

Fintantque

Fin Tableaux.

QUELQUES TYPES DE TRI DANS UN TABLEAU ET LEUR ALGORITHME

Un **algorithme de tri** est, en informatique ou en mathématiques, un algorithme qui permet d'organiser une collection d'objets selon une relation d'ordre déterminée. Trier un tableau c'est donc ranger les éléments d'un tableau en ordre croissant ou décroissant. Dans ce cours on ne fera que des tris en ordre croissant. Il existe plusieurs méthodes de tri qui se différencient par leur complexité d'exécution et leur complexité de compréhension pour le programmeur.

a) TRI PAR INSERTION

En général, le tri par insertion est beaucoup plus lent que d'autres algorithmes comme le tri rapide (ou *quicksort*) et le tri fusion pour traiter de grandes séquences, car sa complexité asymptotique est quadratique. Le tri par insertion est cependant considéré comme le tri le plus efficace sur des entrées de petite taille. Il est aussi très rapide lorsque les données sont déjà presque triées.

Algorithme

```

1: procédure tri_insertion(T)
2: N ← taille de T
3: pour i de 1 à N -1 faire
4:   x ← T[i]
5:   j ← i
6:   tant que j > 0 et T[j -1] > x faire
7:     T[j] ← T[j -1]
8:     j ← j - 1
9:   fin tant que
10:  T[j] ← x
11: fin pour
12: fin procédure

```

b) Tri rapide

Principe de la méthode : On considère un tableau T de N nombres. On désigne le dernier élément du tableau comme pivot, et on va alors parcourir le tableau jusqu'à l'avant-dernier élément pour séparer les éléments strictement inférieurs au pivot et

les éléments supérieurs au pivot, en créant deux nouveaux tableaux. Cette étape est appelée partitionnement. Une fois cette opération effectuée, on peut trier chacun des deux tableaux de manière récursive (la fonction de tri s'appelle elle-même, mais sur des tableaux strictement plus courts), puis reconstituer le tableau trié en concaténant le tableau trié des éléments plus petits que le pivot, le pivot, et le tableau trié des éléments plus grands que le pivot

- ▶ Choisir un élément du tableau appelé *pivot* ;
- ▶ Ordonner les éléments du tableau par rapport au pivot ;
- ▶ Appeler récursivement le tri sur les parties du tableau à droite du pivot.

EXEMPLE D'ALGORITHME

procédure triRapide (E/S t : Tableau[1..MAX] d'Entier; gauche,droit : Naturel)

Déclaration pivot : Naturel

Début

Si gauche<droite alors

Partition(t, gauche,droite,pivot)

triRapide(t,gauche,pivot-1)

triRapide(t,pivot+1,droite)

Finsi

Fin

c) TRI A BULLE

Encore appelé tri par propagation, Il consiste à comparer répétitivement les éléments consécutifs d'un tableau et à les permuter lorsqu'ils sont mal triés

1: procedure tri bulles(T)

2: N ← taille de T

3: pour i de N - 1 à 1 faire

4: pour j de 0 à i ← 1 faire

5: si T[j] > T[j + 1] alors

6: échanger T[j] et T[j + 1]

7: finsi

8: fin pour

9: finpour

10: fin procédure

SITUATION D'INTEGRATION

- 1- Présentez quelques types de recherche dans un tableau que vous connaissez
- 2- Quelle différence faites-vous entre une recherche dichotomique et une recherche séquentielle ?
- 3- Donnez le principe de la recherche linéaire
- 4- Donnez une définition d'algorithme de tri
- 5- Citez les différents algorithmes de tri que vous connaissez

REINVESTISSEMENT

- 1- Identifiez les algorithmes suivants

a)

```
1: procedure tri_insertion(T)
2: N ← taille de T
3: pour i de 1 à N -1 faire
4: x ← T[i]
5: j ← i
6: tant que j > 0 et T[j -1] > x
  faire
7: T[j -1] ← T[j]
8: j ← j - 1
9: fin tant que
10: T[j] ← x
11: finpour
12: finprocédure
```

b)

```
foncti1:      procédure      tri
bulles(T)
2: N ← taille de T
3: pour i de N - 1 à 1 faire
4: pour j de 0 à i — 1 faire
5: si T[j] > T[j + 1] alors
6: échanger T[j] et T[j + 1]
7: finsi
8: finpour
9: finpour
10: finprocédure
```

c)

```
Rech dico(tab : Tableau[0..MAX]
d'Éléments, x : Élément) :Naturel

Déclaration gauche, droit, milieu :
Naturel

Début
gauche←0;droit←MAX
Tant que gauche ≤ droit faire
milieu ← (gauche+droit) div 2
Si x=tab [milieu] alors retourner
milieu finsi
Si x<tab [milieu] alors
Droit ← milieu-1
Sinon
Gauche ← milieu+1
Finsi
Fintantque
Retourner MAX+1
tableau
```